

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema 2: Gestión y Envío de Notificaciones (Acoplamiento de Sistemas)

Problema/ Preocupación de diseño

El **Sistema DashKPI** es responsable de "Enviar y recibir notificaciones" a un **Sistema de notificaciones** externo. La preocupación es el **acoplamiento** entre el **core** del sistema DashKPI y la lógica o tecnología específica del Sistema de notificaciones (e.g., si requiere un formato de mensaje específico, un protocolo particular como SMTP o *WebSockets*, o si el sistema de notificaciones falla). Un alto acoplamiento puede hacer que el DashKPI sea frágil y difícil de migrar a otro servicio de notificaciones.

Orientación de la decisión

Desacoplamiento mediante el Patrón *Gateway o Adapter*. Crear una capa de abstracción (interfaz o *gateway*) que intermedie entre la lógica de negocio del DashKPI y el Sistema de notificaciones. Los elementos que serán afectados son el **Backend (Django)**, el **Módulo de Notificaciones** dentro del DashKPI, y la integración con el **Sistema de notificaciones** externo.

Supuestos

Se asume que el **Sistema de notificaciones** es un servicio o módulo externo al control directo del equipo de DashKPI. Se asume que la generación de la notificación (qué, cuándo y a quién) reside en el DashKPI.

Restricciones

* Las notificaciones deben ser enviadas de forma **confiable** (garantía de entrega). * El envío de notificaciones no debe bloquear la ejecución de la lógica de negocio principal.

Requerimientos relacionados

Configuración de Notificaciones: Los usuarios deben poder configurar sus "notificaciones preferidas". **Funcionalidad:** El sistema debe ser capaz de enviar y recibir notificaciones.

Alternativas consideradas

Alternativa 1 Acoplamiento Directo (Sin Abstracción): Las clases del DashKPI invocan directamente la API o los métodos específicos del Sistema de notificaciones.

Alternativa 2 Patrón *Adapter* (Decisión seleccionada): Crear una interfaz genérica *ISender* en DashKPI. Se implementa una clase *NotificacionAdapter* que traduce la llamada genérica del DashKPI al formato y protocolo específico del Sistema de notificaciones.

Alternativa n Comunicación Asíncrona con *Message Broker*: El DashKPI publica un evento genérico de "Notificación Generada" en un *Message Broker* (e.g., Kafka, RabbitMQ). El Sistema de notificaciones consume el mensaje.

Evaluación de
alternativas

Alternativa 1 Ventajas: Implementación rápida y sencilla inicialmente.
Desventajas: Máximo acoplamiento. Cualquier cambio en el servicio externo de notificaciones requiere modificar el *core* de DashKPI.

Alternativa 2 Ventajas: Bajo acoplamiento. El DashKPI solo depende de una interfaz genérica. Facilita la migración a un nuevo sistema de notificaciones.
Desventajas: Se añade una capa de código (el *Adapter*).

Alternativa n Ventajas: Máximo desacoplamiento. El envío es asíncrono y tolerante a fallos. **Desventajas:** Introduce complejidad (un nuevo *broker* de mensajes) y puede ser excesivo si solo hay un Sistema de Notificaciones.

Alternativa
seleccionada y
Justificación

Alternativa 2: Patrón Adapter. Ofrece un **excelente equilibrio** entre desacoplamiento y complejidad. Permite que la lógica del DashKPI genere notificaciones en un formato interno, mientras que el *Adapter* se encarga de las peculiaridades del sistema externo, cumpliendo el requisito de "Enviar y recibir notificaciones" de forma flexible.

Decisiones
relacionadas

El envío de notificaciones será procesado de forma **asíncrona** (a través de *threads* o una cola liviana interna) para evitar bloquear el hilo de ejecución principal del DashKPI, respetando la restricción de rendimiento.

Comentarios

El adaptador debe manejar la lógica para las **notificaciones preferidas** del Miembro de equipo, como si desea recibirlas por email o *push* (si el Sistema de Notificaciones lo soporta).